University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: **AI-Driven Image Recognition for Automated Plant Disease Diagnosis in Modern Smart Farming**

Programme Title: **MSc Data Analytics**

Name: **Aadhithya Roy Payaroli**

Year: **2026**

CONTENTS

CONTENTS 2

ABSTRACT 4

1. INTRODUCTION 5

2. LITERATURE REVIEW 6

3. PROBLEM FORMULATION 8

4. DATASET PREPARATION 9

5. MODEL ARCHITECTURES 12

6. IMPLEMENTATION (PYTHON + TENSORFLOW) 16

7. MODEL EVALUATION 17

8. CONCLUSION 25

9. REFERENCES 26

10. LIST OF TABLES AND FIGURES 27

APPENDIX 28

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters): AADHITHYA ROY PAYAROLI

Date: 06 – February – 2026

ABSTRACT

This project develops an automated system for classifying diseases in plant leaves using deep learning, with the aim of helping farmers detect problems early in agriculture.

Three models were built and compared. A custom convolutional neural network (CNN) created as the baseline, MobileNetV2 with transfer learning (ImageNet weights kept frozen), and ResNet50 with transfer learning (ImageNet weights kept frozen). The performance of each model was measured. Using accuracy, along with macro-averaged precision, recall, and F1-score. Confusion matrices were generated and selected example predictions were also reviewed.

The models were evaluated for practical deployment by considering the number of parameters, the size of the model, and inference latency. The findings highlight the trade-off between classification accuracy and computational efficiency — a factor that matters greatly for real-world use in agriculture.

1. INTRODUCTION

Many plant diseases show early symptoms that are small and look similar to each other. Because of this, even experienced farmers and agriculture experts find it difficult to identify them correctly just by looking (Shoaib et al., 2022). In large farms, checking every plant regularly takes a lot of time and workers. It is often not practical. Correct diagnosis at an early stage helps apply treatment only where needed. It stops the disease from spreading and also reduce harm to the environment (Dhaka et al., 2021). Traditional methods of disease identification depend heavily on expert knowledge. These methods are difficult to use across many crops, different regions, and various farming conditions (Shoaib et al., 2022).

In recent years, deep learning has shown good results in solving such problems. Convolutional Neural Networks (CNNs) perform very well in image classification tasks. They can learn important visual features directly from raw images without needing manually designed features (Pandian et al., 2022). When applied on plant leaf images, these models can recognise disease patterns. Such as changes in colour, texture, and shape of spots or lesions (Sladojevic et al., 2016). The availability of large labelled image datasets in agriculture has also helped build reliable systems based on data (Chowdhury et al., 2021). However, using deep learning models in actual farming conditions brings some practical challenges. Many high-accuracy models need large amounts of computing power. They are slow to run and large in size. Therefore, such models are not suitable for mobile phones or low-cost edge devices that farmers commonly use in the field (Too et al., 2019).

The main goal of this study is to build and test an automated system that classifies plant leaf diseases using deep learning on images. This study compares several deep learning architectures using a standard plant leaf disease dataset. The main focus is to identify a suitable balance between classification accuracy, inference time (speed of prediction), and model size (Pandian et al., 2022).

By evaluating both classification performance and practical efficiency aspects, the work aims to contribute to the development of realistic and scalable solutions for automated plant disease diagnosis in precision agriculture (Dhaka et al., 2021).

2. LITERATURE REVIEW

Over the past decade, computer vision techniques for detecting plant diseases have advanced significantly (Dhaka et al., 2021). Early studies mainly relied on traditional image processing methods. These approaches involved manually extracting features from leaf images, including colour histograms, texture descriptors, and shape-related measurements. These extracted features were then used as input to traditional machine learning classifiers, such as Support Vector Machines (SVM) or k-Nearest Neighbours (k-NN) (Sladojevic et al., 2016). These approaches produced acceptable results under controlled laboratory conditions. However, they showed major limitations in real-world field conditions.

The introduction of deep learning brought a significant improvement in plant disease classification. It can automatically learn layerred (hierarchical) features straight from the raw images (Pandian et al., 2022). Studies that applied standard CNN architectures have consistently reported high classification accuracy for diagnosing plant diseases (Chowdhury et al., 2021). Like ImageNet and fine-tune them for plant diseases. These results clearly show that deep learning is a very effective. Architectures such as ResNet (deep residual networks) and MobileNet (lightweight networks for mobile use) are common. Transfer learning trains faster, gives higher accuracy, and generalises better than training from zero. This is useful when labelled plant images are few. Now, transfer learning is the main method in this field (Too et al., 2019). Still, some problems exist. Most studies focus only on accuracy. They ignore model size, inference time, or computing needs. Large models use much memory and power. This makes them hard to run on mobile phones or edge devices in farms. Also, tests often use clean datasets from labs. This can give results that do not match real field conditions (Too et al., 2019).

There is a common limitation in the existing literature. There is the lack of direct and systematic comparisons between deep learning models that differ in size and complexity. Lightweight models typically have fewer parameters and provide faster inference times. However, most studies do not clearly report or quantify the drop in classification accuracy that occurs in exchange for these improvements in speed and efficiency. This makes it difficult to determine which models offer the most practical trade-off between performance and resource requirements (Pandian et al., 2022).

This study fills these gaps. It compares different deep learning architectures on classification accuracy, inference time, and model size. By looking at both performance and efficiency, the work helps build practical systems for plant disease detection that can scale in real agriculture (Dhaka et al., 2021).

3. PROBLEM FORMULATION

In many farming areas, disease identification still depends on manual inspection by farmers or experts. This method takes a lot of time. It is especially difficult in regions where agricultural experts are few (Shoaib et al., 2022). Because of late detection, diseases often cause visible damage before they are noticed. Which leads to lower crop yield and higher financial losses (Chowdhury et al., 2021).

From a technical point of view, this task is a multi-class image classification problem. The aim is to take a plant leaf image and assign it to one specific class. Either a particular disease or the healthy category. Each image in the dataset has a label that indicates the crop type and the disease (or healthy status). The main difficulty is to learn visual patterns that can separate diseases even when their symptoms look very similar (Pandian et al., 2022). Many plant disease diagnosis systems need to work on devices with limited resources, such as smartphones or low-cost edge devices used in the field. These devices have restricted processing power, memory, and battery life (Too et al., 2019). Although large deep learning models often give good accuracy, they usually have too many parameters and take too long to make predictions, which makes them unsuitable for such devices (Too et al., 2019). Therefore, this study addresses two connected goals:

- to build a system that classifies plant leaf diseases accurately, and
- to find a model that also performs well in terms of computational efficiency.

The work compares different deep learning architectures with varying levels of complexity. The main objective of this study is to find out which model provides the best balance between classification accuracy, inference speed (time taken to make one prediction), and model size (number of parameters and storage space required) (Pandian et al., 2022). Formally, given an input RGB leaf image $x \in R^{H \times W \times 3}$, the model predicts a class $y \in \{1,\ldots,C\}$, where C is the total number of classes (including all diseases and the healthy class). The training objective is to minimise the multi-class cross-entropy loss over the training dataset.In addition to classification accuracy, this study also evaluates practical factors related to real-world deployment. These include the number of model parameters, the model size, and inference latency measured on standard hardware (Dhaka et al., 2021).

4. DATASET PREPARATION

4.1. Source

This project uses the PlantVillage leaf image dataset. This is publcly available and is widely used (Hughes and Salathé, 2015). The PlantVillage dataset contains 54,305 images of plant leaves, covering both healthy and diseased samples. All images were captured under controlled conditions. It includes 14 crop species — such as apple, blueberry, cherry, grape, orange, peach, pepper, potato, raspberry, soybean, squash, strawberry, and tomato — along with multiple disease categories. Healthy leaf images are also included for many of these crops (Mohanty et al., 2016).

4.2. Labels

The dataset is organised using a folder-based structure. Each folder name represents one class (a specific crop–disease combination or healthy leaves). This project treats the task as multi-class image classification, where each image belongs to exactly one class (Too et al., 2019).

4.3. Acknowledgement

The dataset was originally collected and curated by the PlantVillage project. It has been used in many early deep learning studies on plant disease recognition (for example, Mohanty et al., 2016). In this work, the version from the Kaggle link above is used to support reproducibility.

4.4. Subset selection (10-class setup)

To keep the training time reasonable while still maintaining a meaningful multi-class classification problem, we use only 10 classes. The selected subset includes a total of 7,415 images.

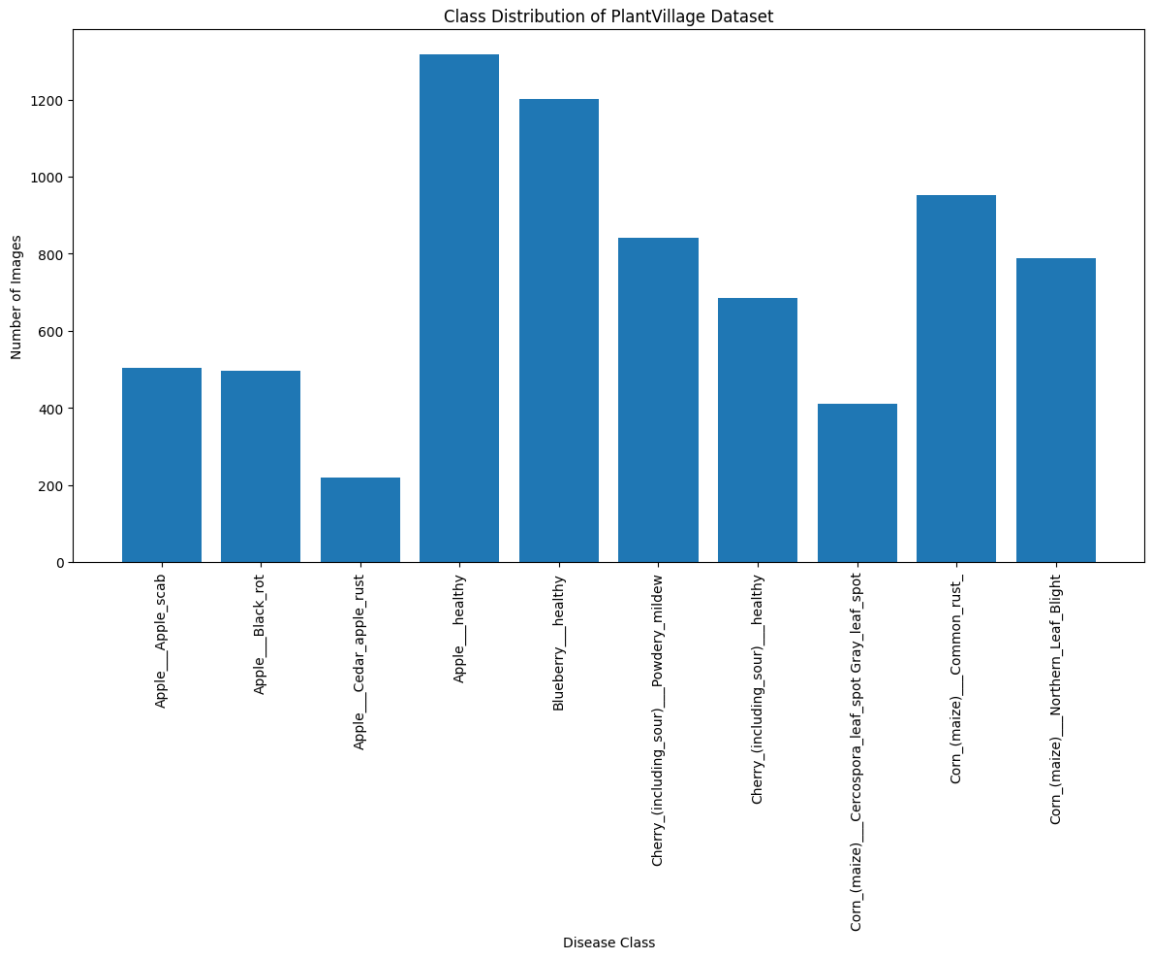

Figure 1: Class Distribution of PlantVillage Dataset (only 10 used)

4.5. Train/Validation/Test split

- Training set: 5,189 images (70%)
- Validation set: 1,113 images (15%)
- Test set: 1,113 images (15%)

The training set is used to model train. The validation set is used to adjust hyperparameters. Check the model's learning progress during training. Make decisions such as early stopping to avoid overfitting. The test set is to measure the model's final performance.

4.6. Preprocessing (applied to all models)

All images are resized to 224 × 224 pixels to match the input size expected by the deep learning architectures used in this study. Pixel value normalisation is performed to help training. For the

custom baseline CNN: pixel values are rescaled to the range [0, 1] by dividing by 255. For transfer learning models (MobileNetV2 and ResNet50): the official preprocess_input function of each model is used to match the preprocessing applied during their original ImageNet training.

4.7. Training-time data augmentation

To reduce overfitting and make the models more robust to small changes in leaf images, augmentation is applied only to the training set. The following transformations are used:

- Random horizontal and vertical flips
- Random rotation (up to 0.05 radians, i.e., about ±3 degrees)
- Random zoom (up to 10%)

These augmentations increase the effective variety of training data while keeping important disease features (such as lesion shape, colour, and texture) intact.

4.8. Reproducibility and fair comparison

All steps are applied in exactly the same way for every experiment. The differences in model performance come mainly from the choice of architecture and its computational characteristics, and not from variations in how the data is prepared.

5. MODEL ARCHITECTURES

These models represent different design approaches — from a simple baseline to lightweight and high-capacity architectures — which allows a fair and systematic comparison of both performance and computational efficiency.

5.1. Custom Convolutional Neural Network (Baseline)

Three convolution blocks + max-pooling layers. The number of filters increases progressively: 32 → 64 → 128. Each convolution layer uses ReLU activation. After the convolution blocks, GlobalAveragePooling2D is applied, followed by dropout for regularisation and a final dense layer with softmax activation for 10-class prediction.

Global average pooling is used instead of a large flatten + dense layer. This reduces the number of parameters and gives a clean, simple baseline to compare against transfer learning models.

This custom CNN has a small number of parameters and very fast inference time, which makes it computationally efficient. However, because of its limited depth and capacity, it may struggle to learn complex disease patterns, especially when classes have very similar visual features.

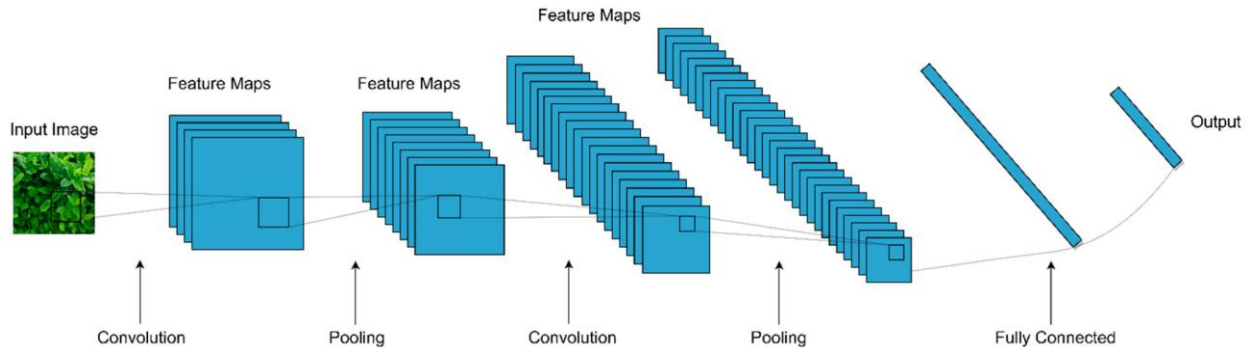

Figure 2: Structure of a typical CNN model [Source: (Dhaka et al., 2021)].

```python
def build_custom_cnn(num_classes):
    inputs = tf.keras.Input(shape=(*IMG_SIZE, 3))
    x = data_augmentation(inputs)
    x = rescale(x)

    x = tf.keras.layers.Conv2D(32, 3, activation="relu", padding="same")(x)
    x = tf.keras.layers.MaxPool2D()(x)

    x = tf.keras.layers.Conv2D(64, 3, activation="relu", padding="same")(x)
    x = tf.keras.layers.MaxPool2D()(x)

    x = tf.keras.layers.Conv2D(128, 3, activation="relu", padding="same")(x)
    x = tf.keras.layers.MaxPool2D()(x)

    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.3)(x)

    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)
    return tf.keras.Model(inputs, outputs, name="custom_cnn")
```

5.2. MobileNetV2 (Lightweight Architecture)

The model uses depthwise separable convolutions and inverted residual blocks. These techniques greatly reduce the number of parameters and the amount of computation compared to standard CNNs. In this study, MobileNetV2 is used with transfer learning. The model is loaded with pre-trained weights from ImageNet (include_top=False), and the backbone layers remain frozen during training. A simple classification head is added on top. It consists of a GlobalAveragePooling2D layer, followed by one or more dense layers, and finally a softmax output layer for the 10 classes.

Only the newly added classification head is trained on the PlantVillage subset. Freezing the backbone reduces training time and decreases the risk of overfitting, which is especially helpful when working with the smaller 10-class dataset. At the same time, it allows the model to use powerful, general visual features that were already learned from the large ImageNet dataset.

This architecture is well suited for real-world deployment in situations where low inference latency and small model size are important requirements.

```python
def build_mobilenetv2(num_classes):
    inputs = tf.keras.Input(shape=(*IMG_SIZE, 3))
    x = data_augmentation(inputs)
    x = mobilenet_preprocess(x)

    base = MobileNetV2(include_top=False, weights="imagenet", input_shape=(*IMG_SIZE, 3))
    base.trainable = False

    x = base(x, training=False)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)

    model = tf.keras.Model(inputs, outputs, name="mobilenetv2_tl")
    return model
```

5.3. ResNet50 (High-Capacity Architecture)

It uses residual (skip) connections that allow very deep networks to be trained effectively. These skip connections help solve the vanishing gradient problem. Which commonly occurs, when training deep neural networks. As a result, the model can learn more detailed and hierarchical features. That leads to better classification performance in most cases.

In this study, ResNet50 is also implemented using transfer learning. The model is loaded with pre-trained ImageNet weights (include_top=False). The backbone layers remain frozen, and only a lightweight classification head is added and trained on the PlantVillage subset data. ResNet50 is expected to give high classification accuracy because of its greater representational power. However, it has a much larger number of parameters and longer inference time compared to MobileNetV2. Computational cost increases with model complexity.

```python
def build_resnet50(num_classes):
    inputs = tf.keras.Input(shape=(*IMG_SIZE, 3))
    x = data_augmentation(inputs)
    x = resnet_preprocess(x)

    base = ResNet50(include_top=False, weights="imagenet", input_shape=(*IMG_SIZE, 3))
    base.trainable = False

    x = base(x, training=False)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)

    model = tf.keras.Model(inputs, outputs, name="resnet50_tl")
    return model
```

5.4. Advantages and Disadvantages

Table 1: Advantages and Disadvantages of each model

Model	Advantages	Disadvantages
Custom CNN (Baseline)	• The model is simple to design, train, and understand. It serves as a good basic reference. • It has fewer parameters than large pre-trained models, Inference is fast and requires very little computing power	• It has lower capacity to learn complex patterns. Because of this, it can confuse diseases that have very similar visual appearance. • It usually requires more careful tuning of hyperparameters to reach the same level of accuracy as models that use transfer learning.
MobileNetV2 (Frozen backbone + head)	• It gives a good balance between accuracy and efficiency. It was made for mobile phones and edge devices. • It uses features already learned from ImageNet, low number of parameters and small model size	• The classes have very small visual differences. • Because the backbone remains frozen, the model cannot adapt much to the new task.
ResNet50 (Frozen backbone + head)	• It can learn strong and detailed features. This allows it to achieve high accuracy through out different classes. • Residual connections makes the model learn deep and useful patterns effectively.	• The model size is large, which increases storage needs and makes deployment harder. • It needs more computing power, which can reduce speed in real-time applications.

Choosing these three architectures, which have different designs and levels of complexity, allows the study to clearly check how increasing model complexity changes classification performance and use of computing resources. This matches the main goal of the research.

6. IMPLEMENTATION (PYTHON + TENSORFLOW)

In this project, Python serves as the main programming language. TensorFlow and Keras are used to handle the deep learning tasks.

Images are loaded into tf.data datasets. During loading, the data passes through preprocessing steps. For the training set, data augmentation is applied on the fly while forming batches.

Training follows a standard supervised learning approach. The loss function is categorical cross-entropy. The model parameters are updated by Adam optimiser. Early stopping monitors validation performance. Which helps prevent overfitting.

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.1),
], name="augmentation")
```

MobileNetV2 and ResNet50 show up with their convolutional layers preformed by ImageNet. A simple classification layer caps them off. In training the main structure remains locked, just that layer sees tweaks, this reduces processing demands and holds steadiness amid a limited farming data collection.

```
EPOCHS = 25
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=5, restore_best_weights=True)
]
```

Reproducibility counts heavily in this project. The notebook generates model overviews, records parameter totals, lists saved model sizes in megabytes, measures inference speed in milliseconds for each image. Speed checks follow a uniform method: test data group, batch size locked at 32, brief initial run to warm things up. Findings appear as average with standard deviation added or subtracted, milliseconds per image, these mirror the real execution setting, be it CPU or GPU, plus the grouping approach applied.

7. MODEL EVALUATION

Figure 3 to 8 shows, in the baseline CNN, the validation loss has noticeable fluctuations. This points to limited generalisation and some instability during training. MobileNetV2 converges very quickly, usually within just a few epochs. It reaches high accuracy values, with training and validation accuracy staying close to each other. The loss also remains low and stable. That means that the pre-trained ImageNet features transfer good and the model generalises well.

While, ResNet50 achieves the highest training accuracy among the three models. There is a small, but consistent gap between the training and validation curves. This indicates a mild degree of overfitting, even though the model has strong representational capacity.

Overall, the transfer learning models (MobileNetV2 and ResNet50) converge faster and more reliably than the custom baseline CNN.

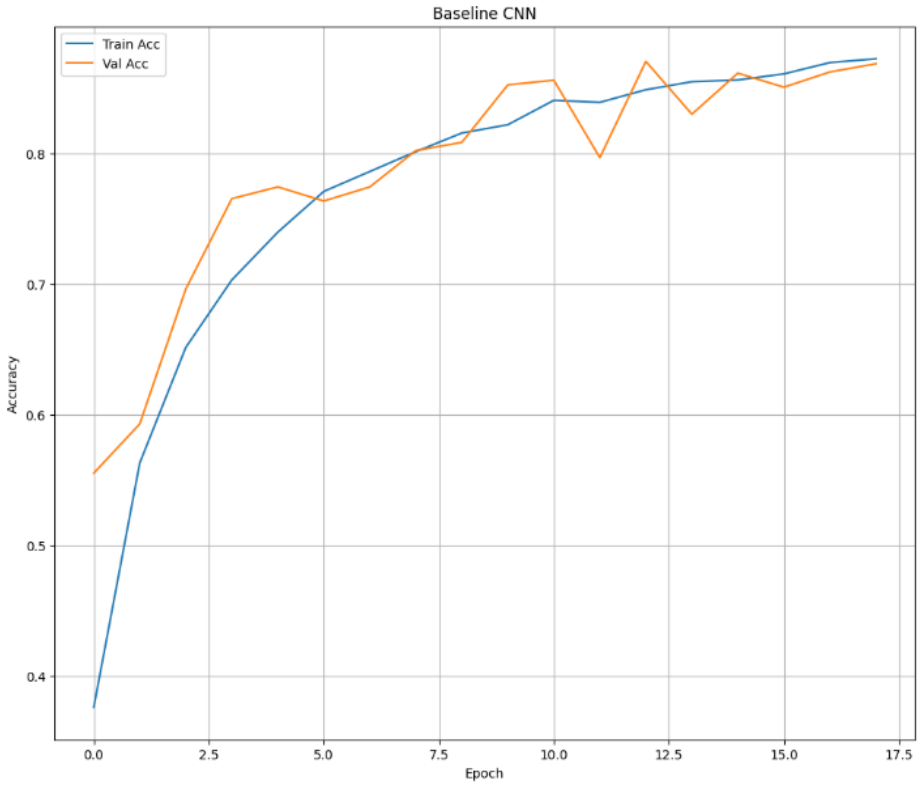

Figure 3: Baseline CNN Epoch vs Accuracy

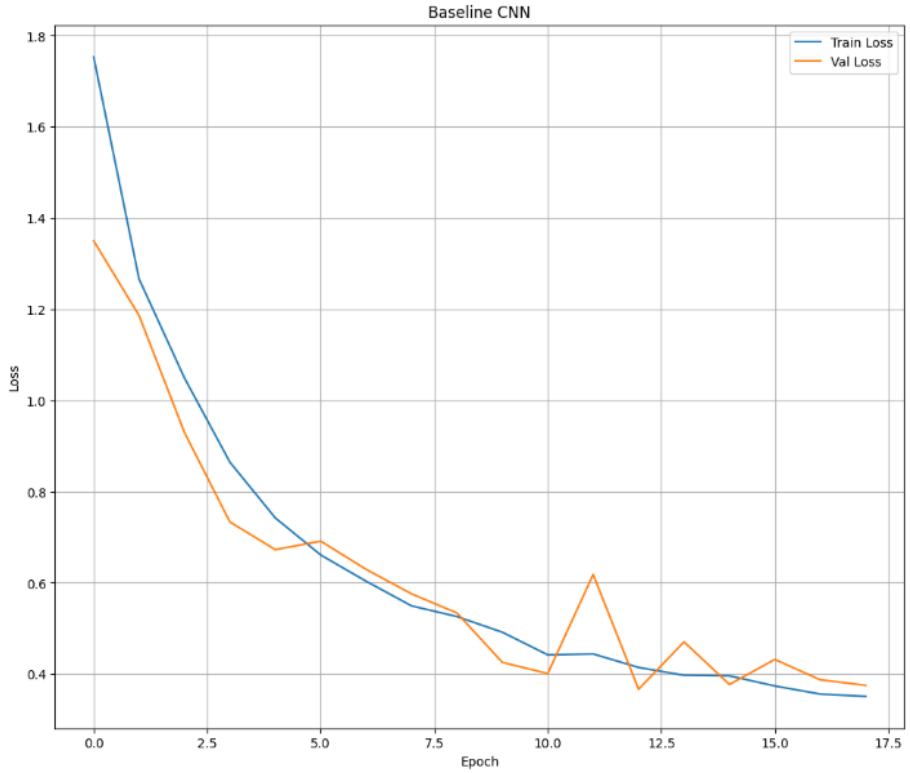

Figure 4: Baseline CNN Epoch vs Loss

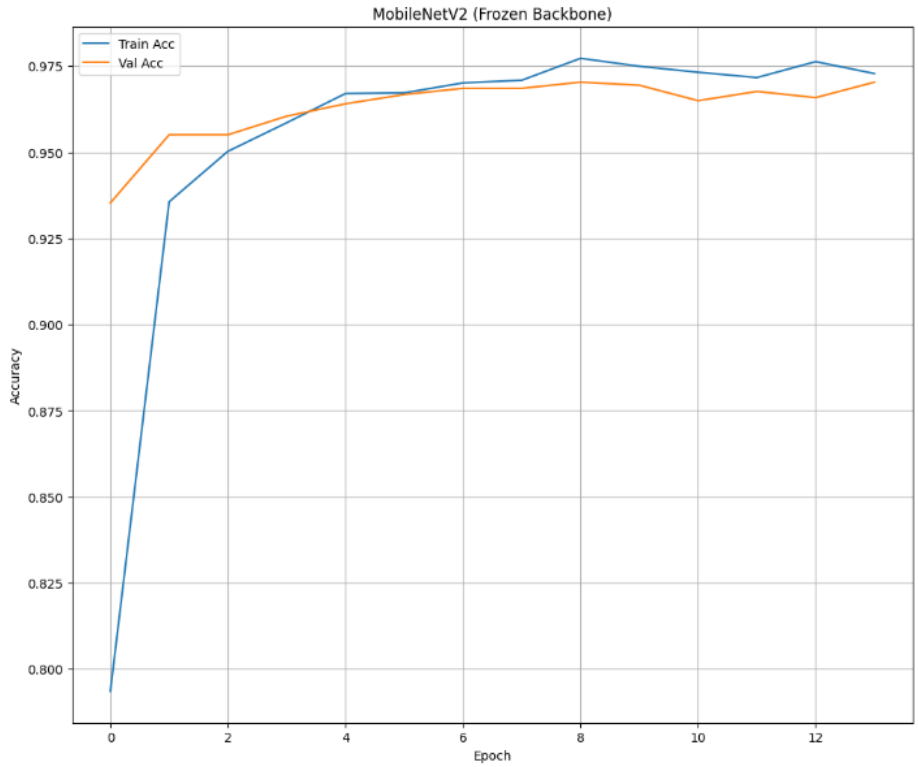

Figure 5: MobileNetV2 (Frozen Backbone) Epoch vs Accuracy

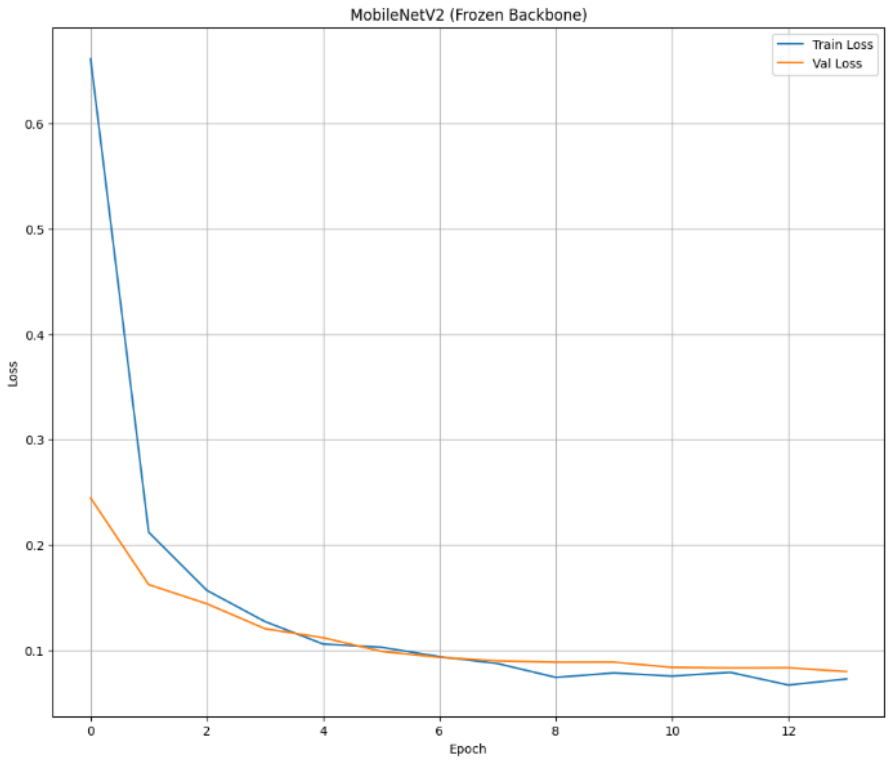

Figure 6: MobileNetV2 (Frozen Backbone) Epoch vs Loss

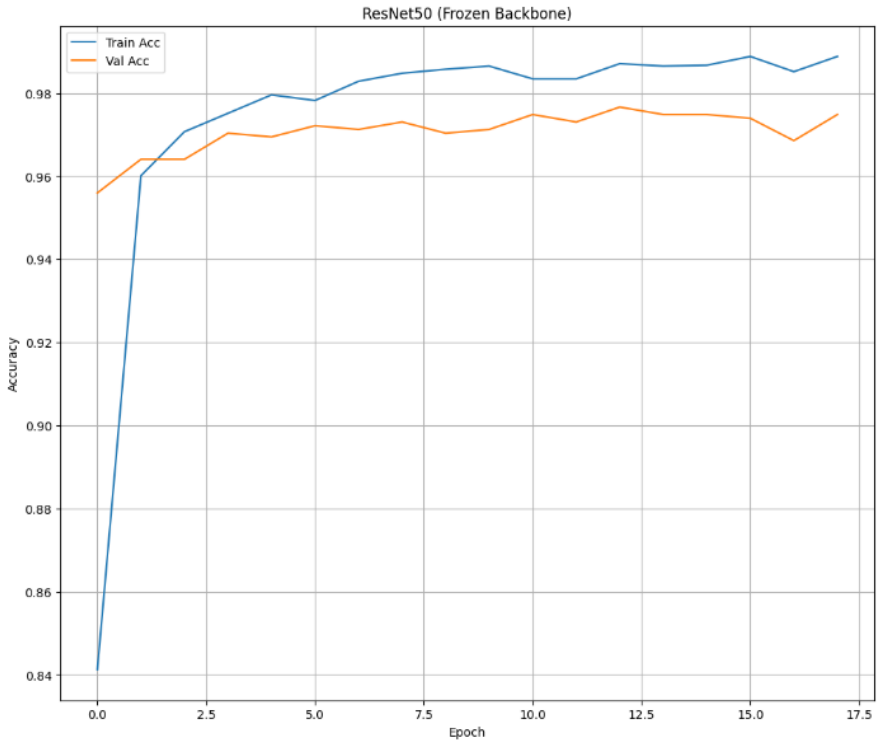

Figure 7: ResNet50 (Frozen Backbone) Epoch vs Accuracy

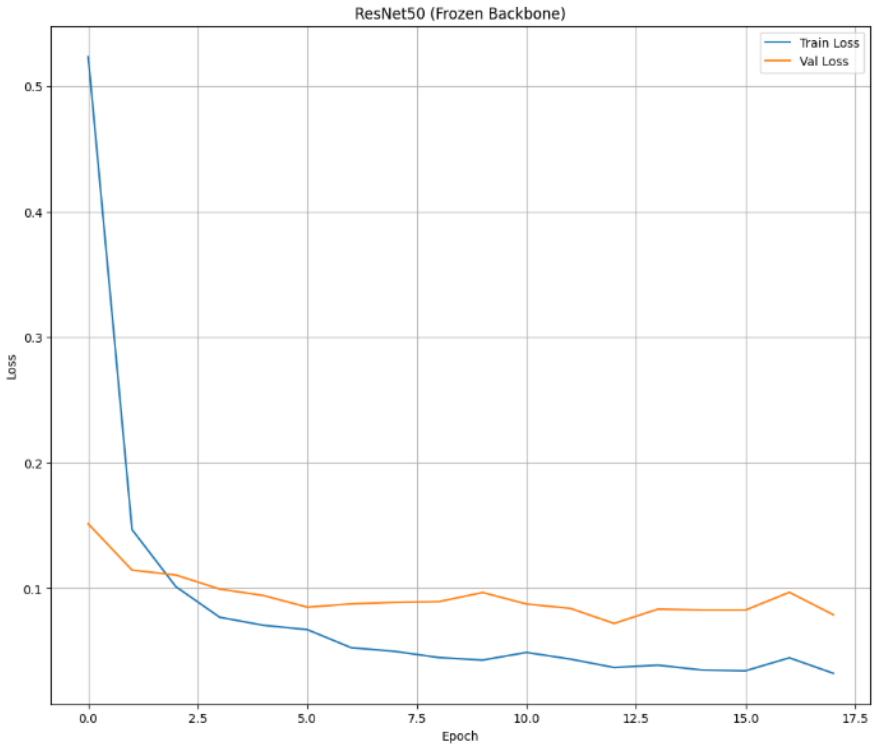

Figure 8: ResNet50 (Frozen Backbone) Epoch vs Loss

We carry out the evaluation on that separate test set we held back. Overall accuracy gets reported, along with those macro-averaged figures for precision, recall, and F1-score, an equal treatment for every class in the mix. To spot those recurring mix-ups among diseases that look alike in images, we turn to confusion matrices normalized by the true class.

The normalised confusion matrices show that all three models have strong values along the main diagonal. This means each model makes mostly correct predictions for each class. The custom CNN has the largest number of off-diagonal entries. This indicates weaker separation between classes, particularly for disease categories that look visually similar.

MobileNetV2 shows a clear reduction in misclassifications compared to the baseline. The remaining errors are mostly found among the maize disease classes. ResNet50 produces the cleanest confusion matrix among the three models.

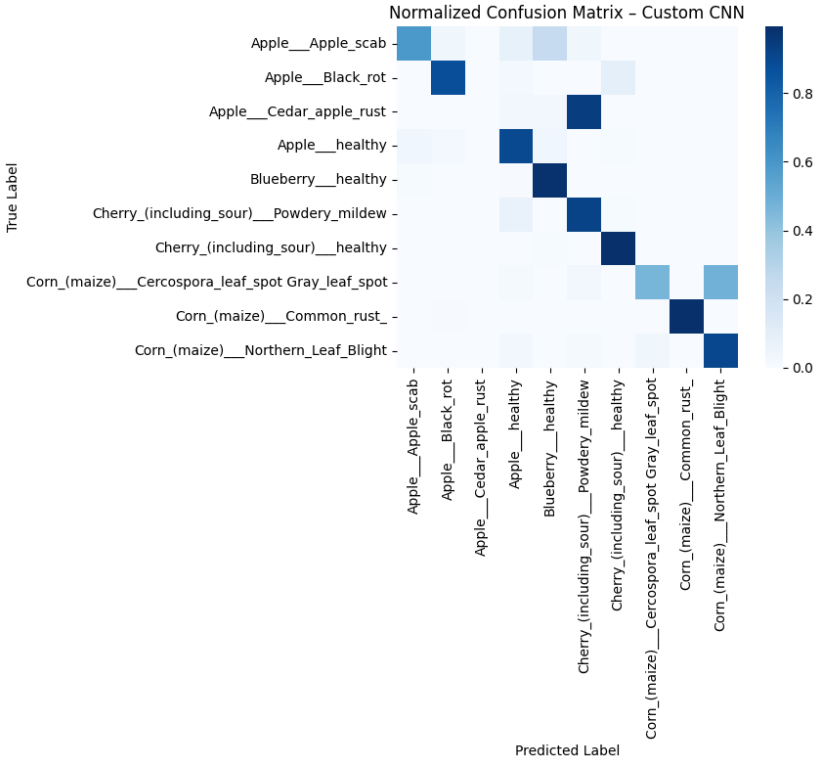

Figure 9: Normalized Confusion Matrix – Custom CNN

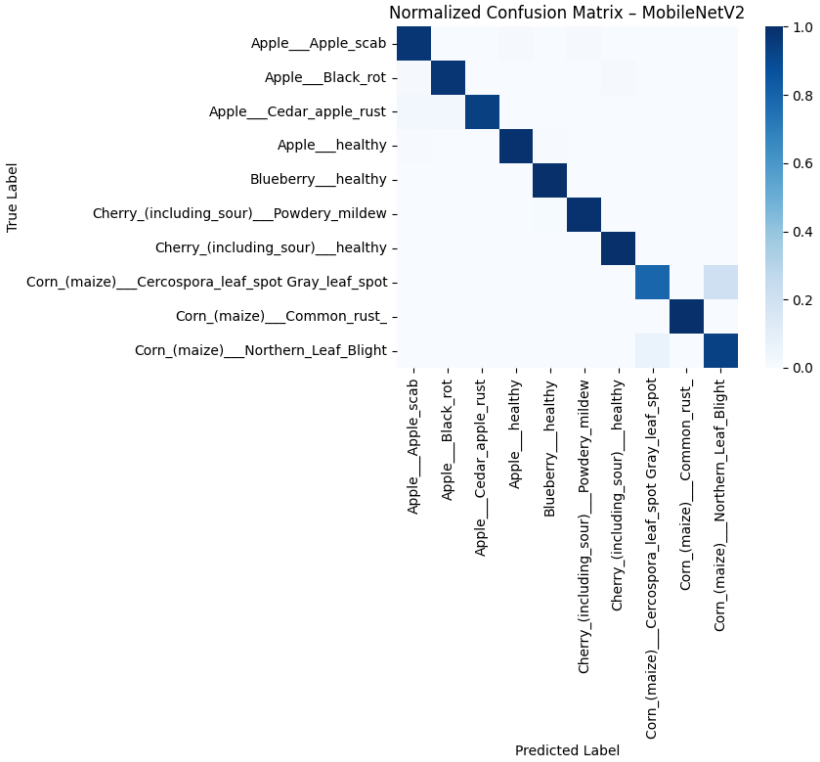

Figure 10: Normalized Confusion Matrix – MobileNetV2

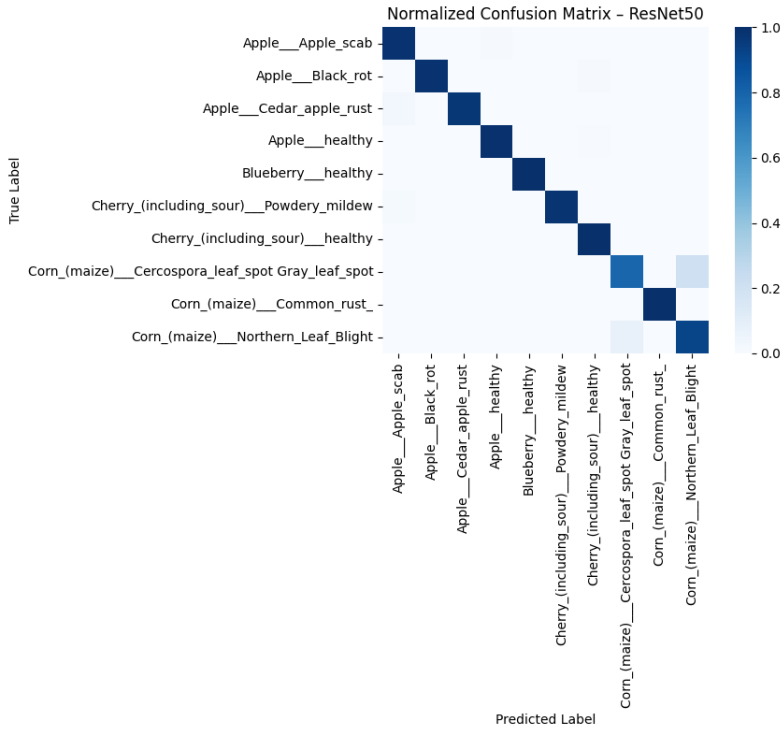

Figure 11: Normalized Confusion Matrix – ResNet50

As shown in Table 2, the custom CNN has very low inference latency and small model size. However, its accuracy and macro F1-score are much lower than the other models. This indicates that it does not generalise well on the test data.Both MobileNetV2 and ResNet50 reach almost the same accuracy, around 97.6%, and macro F1-score above 0.96. This shows that pre-trained features from ImageNet work well on unseen images in this dataset.

Table 2: Model Comparison (Performance vs Scalability) Table

Model	**Input Size**	**Accuracy**	**Macro Precision**	**Macro Recall**	**Macro F1**	**Params**	**Saved Size (MB)**	**Latency (ms/image)**
Custom CNN	224×224	0.8670	0.8579	0.7816	0.7854	94,538	1.14	0.3445 ± 0.0585
MobileNetV2 (TL, frozen)	224×224	0.9757	0.9698	0.9623	0.9655	2,270,794	9.33	4.3915 ± 0.0558
ResNet50 (TL, frozen)	224×224	0.9757	0.9722	0.9643	0.9675	23,608,202	90.86	6.8473 ± 0.0478

Figure 12 shows the trade-off between accuracy and efficiency on the test set. It plots the macro F1-score against the mean inference latency for each model. The plot clearly separates the custom CNN from the two transfer learning models.

MobileNetV2 gives a better balance, it achieves performance very close to ResNet50 but uses far fewer parameters, takes less storage space, and has much lower inference latency. ResNet50 gives the highest overall performance. But because it needs more computation and has larger size, it is less suitable for devices with limited resources or for applications that require real-time prediction.

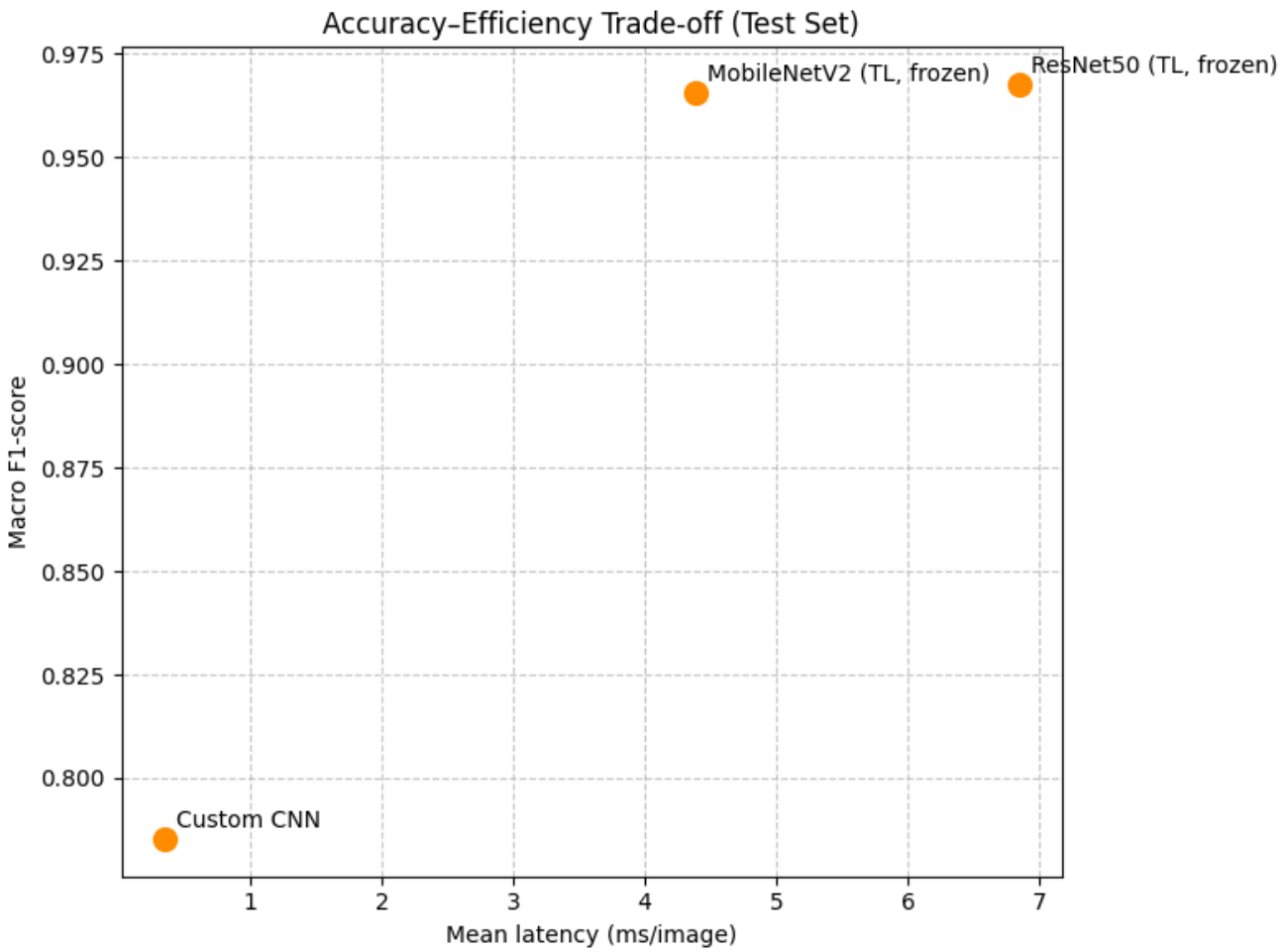

Figure 12: Accuracy–Efficiency Trade-off from the Testing Dataset

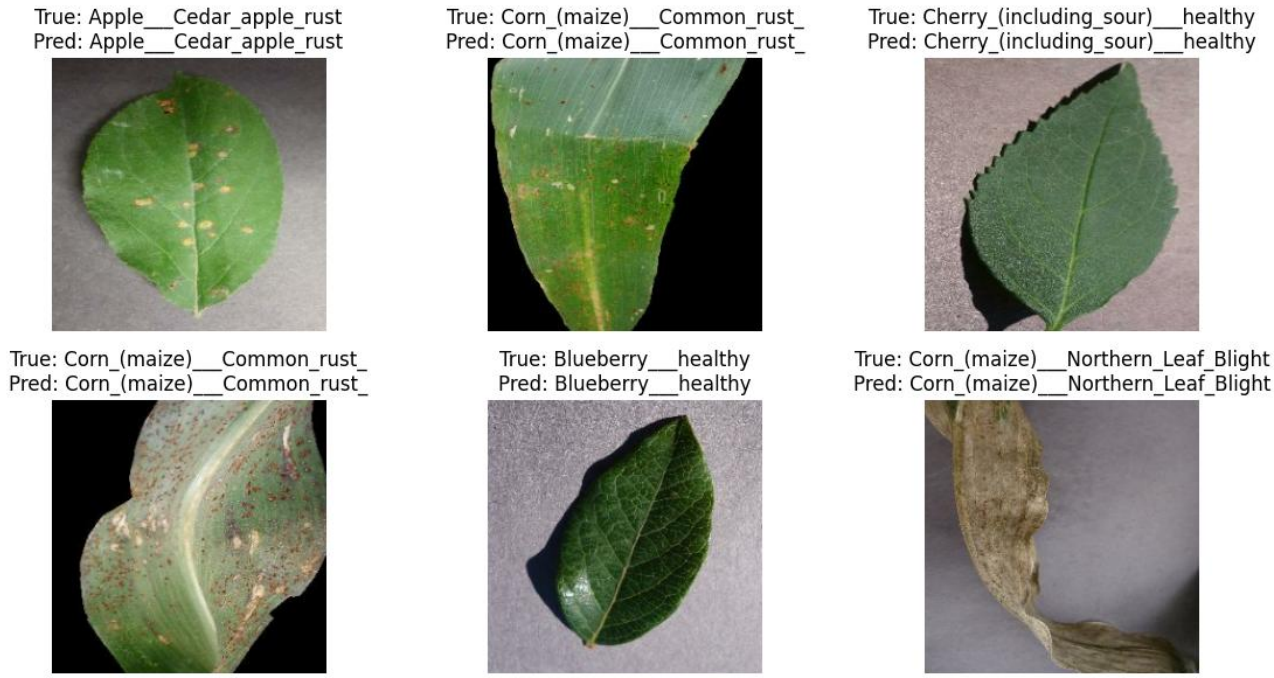

Figure 13: Sample Predictions by MobileNetV2 Model

Figure 13 shows some example predictions made by MobileNetV2 on the test set images that the model had never seen during training. These examples include both correctly classified diseased leaves and healthy leaves.

8. CONCLUSION

This study validates that deep learning is an effective approach for automatic classification of plant leaf diseases, especially when transfer learning is used. Compared to the custom CNN that was trained from scratch, both MobileNetV2 and ResNet50 achieve clearly higher classification performance on the test data. This result confirms that using pre-trained features from ImageNet gives a strong advantage in plant disease recognition tasks. This result confirms that features learned from the large ImageNet dataset help improve accuracy in plant disease recognition.

The study also checked the number of model parameters, the size of the saved model file, and the time taken for inference (making one prediction). ResNet50 gave the best overall classification performance. It uses many more parameters and takes longer to complete predictions. MobileNetV2 achieves almost the same accuracy and macro F1-score as ResNet50. It uses far fewer parameters, requires less storage space, and makes predictions much faster.Because of these reasons, MobileNetV2 is the most suitable choice for devices that have limited resources, such as smartphones or edge devices used in farming.

This study has some limitations. The experiments used only 10 classes from the PlantVillage dataset. Also, all the images were taken under controlled conditions in a lab. In real farm fields, the performance may be lower because of changes in lighting, objects in the background, and different levels of disease severity.

Future work should test the models on more varied datasets collected directly from fields and should include a larger number of disease classes. We can try partial fine-tuning of the pre-trained backbone layers and to apply model compression methods such as quantisation and pruning. These steps can reduce the time and computing resources needed for inference. And still maintaining high accuracy. Such improvements will help create practical and scalable systems for diagnosing plant diseases in real-world precision agriculture applications.

9. REFERENCES

- Dhaka, V.S., Meena, S.V., Rani, G., Sinwar, D., Kavita, K., Ijaz, M.F. and Woźniak, M. (2021) 'A survey of deep convolutional neural networks applied for prediction of plant leaf diseases', *Sensors*, 21(14), 4749.
- Shoaib, M., Hussain, T., Shah, B., Ullah, I., Shah, S.M., Ali, F. and Park, S.H. (2022) 'Deep learning-based segmentation and classification of leaf images for detection of tomato plant disease', *Frontiers in Plant Science*, 13, 1031748.
- Chowdhury, M., Rahman, T., Khandakar, A., Ayari, M., Khan, A.S., Al-Emadi, N., Reaz, M.B.I., Islam, M.T. and Ali, S. (2021) 'Automatic and reliable leaf disease detection using deep learning techniques', *AgriEngineering*, 3(2), pp. 294–312.
- Pandian, J., Kumar, V.D., Geman, O., Hnatiuc, M., Arif, M. and Kanchanadevi, K. (2022) 'Plant disease detection using deep convolutional neural network', *Applied Sciences*, 12(14), 6982.
- Sladojevic, S., Arsenovic, M., Anderla, A., Ćulibrk, D. and Stefanović, D. (2016) 'Deep neural networks based recognition of plant diseases by leaf image classification', *Computational Intelligence and Neuroscience*, 2016, 3289801.
- Too, E.C., Yujian, L., Njuki, S. and Yingchun, L. (2019) 'A comparative study of fine-tuning deep learning models for plant disease identification', Computers and Electronics in Agriculture, 161, pp. 272–279.
- Hughes, D.P. and Salathé, M. (2015) 'An open access repository of images on plant health to enable the development of mobile disease diagnostics', *arXiv preprint arXiv:1511.08060*.
- Mohanty, S.P., Hughes, D.P. and Salathé, M. (2016) 'Using deep learning for image-based plant disease detection', *Frontiers in Plant Science*, 7, 1419.

10. LIST OF TABLES AND FIGURES

Table 1: Advantages and Disadvantages of each model 15
Table 2: Model Comparison (Performance vs Scalability) Table 22

Figure 1: Class Distribution of PlantVillage Dataset (only 10 used) 10
Figure 2: Structure of a typical CNN model [Source: (Dhaka et al., 2021)]. 12
Figure 3: Baseline CNN Epoch vs Accuracy 17
Figure 4: Baseline CNN Epoch vs Loss 18
Figure 5: MobileNetV2 (Frozen Backbone) Epoch vs Accuracy 18
Figure 6: MobileNetV2 (Frozen Backbone) Epoch vs Loss 19
Figure 7: ResNet50 (Frozen Backbone) Epoch vs Accuracy 19
Figure 8: ResNet50 (Frozen Backbone) Epoch vs Loss 20
Figure 9: Normalized Confusion Matrix – Custom CNN 21
Figure 10: Normalized Confusion Matrix – MobileNetV2 21
Figure 11: Normalized Confusion Matrix – ResNet50 22
Figure 12: Accuracy–Efficiency Trade-off from the Testing Dataset 23
Figure 13: Sample Predictions by MobileNetV2 Model 24

APPENDIX

- Complete Notebook is available here: Google Colab
- The source of dataset: Kaggle